LAPORAN

WORKSHOP Mobile Web Server

(TIF310808)

Acara 14

SEMESTER



Nama Tugas

SECURITY, EXCEPTION, LOGGING, PRG, DAN INTEGRASI CRUD LENGKAP

Nama:

Adam Ridho Saifulloh

NIM:

E41250216

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Acara 14 membahas penerapan Security, Exception, Logging, PRG (Post/Redirect/Get), dan integrasi CRUD lengkap pada aplikasi Sistem Informasi Akademik. Implementasi ini melanjutkan arsitektur MVC, Repository Pattern, dan Service Layer dari acara sebelumnya. Fokus praktikum adalah membuat proses pengolahan data lebih aman, mampu menangani kesalahan, mencatat kejadian penting ke log, serta menghindari pengiriman ulang form ketika halaman di-refresh.

1.2 Tujuan

- Menerapkan validasi dan praktik keamanan dasar pada aplikasi web.
- Menerapkan penanganan Exception agar kesalahan aplikasi dapat ditangani dengan baik.
- Menerapkan Logger untuk mencatat kejadian dan error aplikasi.
- Menerapkan pola Post/Redirect/Get pada proses CRUD.
- Mengintegrasikan proses CRUD mahasiswa dengan Service dan Repository.
- Menguji fitur tambah, tampil, ubah, hapus, login, validasi, dan penanganan error.

BAB II ALAT DAN BAHAN

- Laptop/PC
- Visual Studio Code
- XAMPP (Apache dan MySQL)
- PHP
- MariaDB/MySQL
- Web Browser
- Bootstrap
- phpMyAdmin
- Source code Bkpm_Acara_14.zip

BAB III PROSEDUR KERJA

Prosedur kerja berikut menjadi bagian utama laporan. Langkah-langkah disusun berdasarkan materi Acara 14 pada BKPM dan disesuaikan dengan implementasi nyata pada source code Bkpm_Acara_14.zip.

1. Menyiapkan project Acara 14

Membuka folder Bkpm_Acara_14 pada Visual Studio Code. Project merupakan pengembangan dari aplikasi Sistem Informasi Akademik pada acara sebelumnya, sehingga struktur MVC, Repository, dan Service tetap digunakan.

2. Menyiapkan database
Menjalankan Apache dan MySQL melalui XAMPP, kemudian memastikan database akademik dapat diakses melalui phpMyAdmin. Project menyediakan file database/bkpm_acara_7 (1).sql sebagai sumber struktur dan data database.

3. Menyiapkan koneksi database
File app/Database.php digunakan sebagai class koneksi PDO. Konfigurasi database juga terdapat pada config/database.php. Koneksi menggunakan PDO sehingga query pada Repository dapat menggunakan prepared statement.

4. Menerapkan security melalui validasi input
Input mahasiswa tidak langsung disimpan ke database. MahasiswaService melakukan validasi terhadap NIM, nama, email, prodi, dan angkatan. Service juga melakukan pengecekan NIM yang sudah digunakan dan keberadaan prodi sebelum Repository menjalankan proses database.

5. Menerapkan Repository Pattern
MahasiswaRepository menangani query SQL untuk membaca, menambah, mengubah, dan menghapus data mahasiswa. ProdiRepository menangani data program studi. Dengan demikian, Controller dan Service tidak menulis query SQL secara langsung.

6. Menerapkan Service Layer
MahasiswaService menerima MahasiswaRepository dan ProdiRepository melalui constructor. Service menjadi lapisan yang menangani validasi, business logic, hasil operasi CRUD, serta penanganan PDOException.

7. Menerapkan Exception Handling
Operasi create(), update(), dan delete() dibungkus dengan try-catch untuk menangani PDOException. Jika terjadi kesalahan database, Service mengembalikan status gagal dan pesan yang sesuai sehingga aplikasi tidak langsung menampilkan error database mentah kepada pengguna.

8. Membuat Logger
File app/Logger.php digunakan untuk mencatat kejadian aplikasi ke storage/logs/app.log. Logger dapat digunakan untuk membantu proses debugging dan mengetahui kejadian error tanpa harus menampilkan detail internal aplikasi kepada pengguna.

9. Menerapkan PRG (Post/Redirect/Get)
Setelah proses POST berhasil, Controller menggunakan redirect menuju halaman mahasiswa. Flash Message disimpan dalam session sebelum redirect. Dengan pola ini, ketika pengguna melakukan refresh halaman, browser tidak mengirim ulang data POST.

10. Menggunakan Flash Message
Pesan hasil proses CRUD disimpan pada session menggunakan helper setFlash(). Setelah redirect, pesan diambil menggunakan getFlash() dan kemudian dihapus sehingga pesan hanya tampil satu kali.

11. Mengintegrasikan CRUD tampil data
Method index() pada MahasiswaController mengambil keyword pencarian dari URL, memanggil Service, lalu menampilkan app/Views/mahasiswa/index.php. Repository melakukan query untuk mengambil mahasiswa beserta nama prodi.

12. Mengintegrasikan CRUD tambah data
Pengguna membuka form tambah mahasiswa. Setelah form dikirim, Controller memanggil MahasiswaService->create(). Service melakukan validasi. Jika valid, Repository menyimpan data. Setelah berhasil, Controller melakukan redirect ke halaman mahasiswa dan menampilkan Flash Message keberhasilan.

13. Mengintegrasikan CRUD ubah data
Controller mengambil ID mahasiswa, meminta data kepada Service, dan menampilkan form edit. Saat data dikirim, Service melakukan validasi kembali kemudian Repository menjalankan update. Jika berhasil, Controller melakukan redirect.

14. Mengintegrasikan CRUD hapus data
Controller menerima ID dari form POST dan memanggil Service->delete(). Service memastikan data tersedia, kemudian Repository menjalankan delete. Hasil proses diteruskan sebagai Flash Message melalui redirect.

15. Menerapkan error page
Project menyediakan app/Views/errors/500.php sebagai halaman ketika terjadi kesalahan server. Penanganan ini membantu memberikan tampilan error yang lebih terkontrol dibandingkan menampilkan detail exception kepada pengguna.

16. Mengatur Front Controller dan routing
public/index.php menjadi entry point aplikasi. File ini membuat dependency Database, Repository, Service, dan Controller, kemudian menentukan proses berdasarkan URL. public/.htaccess membantu mengarahkan request ke index.php.

17. Menguji aplikasi
Aplikasi dijalankan melalui browser. Pengujian dilakukan pada login, tampil data, pencarian, tambah, edit, hapus, validasi input, NIM duplikat, data tidak ditemukan, Flash Message, redirect setelah POST, dan pencatatan log.

3.1 Alur Kerja Acara 14
Browser → public/index.php → Controller → Service → Repository → Database. Jika proses POST berhasil, Controller menyimpan Flash Message → Redirect → GET → View. Jika terjadi exception, error ditangani dan dicatat pada Logger.

BAB IV SOURCE CODE
Source code pada bab ini diambil langsung dari file Bkpm_Acara_14.zip yang diberikan. Kode tidak dibuat ulang dari contoh modul, tetapi didokumentasikan berdasarkan isi ZIP.

app/Database.php

```php
<?php

// Class yang bertugas mengelola koneksi ke database (PDO).
// Objek Database ini nanti "disuntikkan" ke Repository lewat constructor.
class Database
{
    private ?PDO $pdo = null;

    // Koneksi baru dibuat saat pertama kali dibutuhkan
    public function getConnection(): PDO
    {
        if ($this->pdo === null) {
            $config = require __DIR__ . '/../config/database.php';

            $dsn =
"mysql:host={$config['host']};dbname={$config['dbname']};charset={$config['c
harset']}";

            $this->pdo = new PDO($dsn, $config['username'],
$config['password'], [
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES   => false,
            ]);
        }

        return $this->pdo;
    }
}
```

app/Logger.php

```php
<?php

// Mencatat kejadian penting/error ke storage/logs/app.log.
// Detail teknis hanya masuk ke log, TIDAK ditampilkan ke pengguna.
// Jangan pernah menulis password atau data sensitif ke log.
class Logger
{
    public static function error(string $context, ?Throwable $e = null):
void
    {
        $line = date('Y-m-d H:i:s') . ' - [ERROR] ' . $context;

        if ($e !== null) {
            $line .= ' | ' . get_class($e) . ': ' . $e->getMessage()
                  . ' (' . basename($e->getFile()) . ':' . $e->getLine() .
')';
```

```
        }

        self::tulis($line);
    }

    private static function tulis(string $line): void
    {
        $dir = __DIR__ . '/../storage/logs';

        if (!is_dir($dir)) {
            @mkdir($dir, 0775, true);
        }

        // Hapus baris baru agar satu kejadian = satu baris log
        $line = str_replace(["\r", "\n"], ' ', $line);

        // Jika log gagal ditulis, jangan sampai membuat aplikasi ikut error
        @error_log($line . PHP_EOL, 3, $dir . '/app.log');
    }
}
```

app/helpers.php

```
<?php

// Escape output agar aman dari XSS
function e($value): string
{
    return htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8');
}

// Buat URL dari path, contoh: url('/mahasiswa')
function url(string $path = ''): string
{
    return BASE_PATH . $path;
}

// ---------- Flash Message ----------
// Simpan pesan sementara di session. $type = kelas Bootstrap: success /
danger / info
function setFlash(string $type, string $message): void
{
    $_SESSION['flash'] = ['type' => $type, 'message' => $message];
}

// Ambil pesan lalu langsung dihapus dari session, jadi hanya tampil satu
kali
function getFlash(): ?array
{
```

```
    $flash = $_SESSION['flash'] ?? null;
    unset($_SESSION['flash']);

    return $flash;
}

// Pindah halaman, pesan (opsional) disimpan sebagai flash message
function redirect(string $path, string $message = '', string $type =
'success'): void
{
    if ($message !== '') {
        setFlash($type, $message);
    }
    header('Location: ' . url($path));
    exit;
}

// Halaman yang butuh login: kalau belum login, arahkan ke /login
function cekLogin(): void
{
    if (empty($_SESSION['logged_in'])) {
        redirect('/login');
    }
}
```

app/Controllers/BaseController.php

```
<?php

// Induk semua Controller (Inheritance).
// Method yang dipakai bersama (view & redirect) ditulis sekali di sini,
// lalu diwarisi oleh controller lain lewat "extends BaseController".
abstract class BaseController
{
    // Tampilkan file view. Isi $data menjadi variabel di view,
    // contoh: ['daftar' => $x] -> di view bisa langsung memakai $daftar
    protected function view(string $view, array $data = []): void
    {
        extract($data);
        require __DIR__ . '/../Views/' . $view . '.php';
    }

    // Pindah halaman, pesan (opsional) disimpan sebagai flash message di
session
    protected function redirect(string $path, string $message = '', string
$type = 'success'): void
    {
        if ($message !== '') {
            setFlash($type, $message);
```

```
        }
        header('Location: ' . url($path));
        exit;
    }
}
```

app/Controllers/AuthController.php

```
<?php

require_once __DIR__ . '/BaseController.php';

class AuthController extends BaseController
{
    public function loginForm(): void
    {
        // Kalau sudah login, tidak perlu lihat form login lagi
        if (!empty($_SESSION['logged_in'])) {
            $this->redirect('/mahasiswa');
        }

        $this->view('auth/login');
    }

    public function login(): void
    {
        $username = $_POST['username'] ?? '';
        $password = $_POST['password'] ?? '';

        // Login sementara menggunakan data tetap (hardcode)
        if ($username === 'admin' && $password === '12345') {
            session_regenerate_id(true);
            $_SESSION['logged_in'] = true;
            $_SESSION['user_name'] = 'Admin';

            $this->redirect('/mahasiswa', 'Selamat datang, Admin.');
        }

        $this->redirect('/login', 'Username atau password salah.',
'danger');
    }

    public function logout(): void
    {
        unset($_SESSION['logged_in'], $_SESSION['user_name']);

        $this->redirect('/login', 'Anda telah logout.', 'info');
    }
}
```

```
}
```

app/Controllers/MahasiswaController.php

```php
<?php

require_once __DIR__ . '/BaseController.php';
require_once __DIR__ . '/../Logger.php';
require_once __DIR__ . '/../Services/MahasiswaService.php';

// Controller tipis: hanya menerima request, memanggil Service, lalu
menentukan response.
// Tidak ada validasi dan tidak ada query database di sini.
// Error saat membaca data ditangkap (try-catch) di sini: detail ke app.log,
// pengguna hanya mendapat pesan yang aman lewat flash message.
class MahasiswaController extends BaseController
{
    // Constructor injection: Service diberikan dari luar (lihat
public/index.php)
    public function __construct(private MahasiswaService $service) {}

    public function index(): void
    {
        $keyword = trim($_GET['q'] ?? '');

        try {
            $daftar = $this->service->all($keyword);
        } catch (Throwable $e) {
            Logger::error('Gagal memuat daftar mahasiswa', $e);
            setFlash('danger', 'Data mahasiswa gagal dimuat. Silakan coba
lagi nanti.');
            $daftar = [];
        }

        $this->view('mahasiswa/index', ['daftar' => $daftar, 'keyword' =>
$keyword]);
    }

    public function create(): void
    {
        $this->tampilForm(new Mahasiswa(), url('/mahasiswa/store'));
    }

    // POST -> proses -> redirect -> GET (PRG): jika berhasil, pengguna
dialihkan
    // sehingga refresh halaman tidak mengirim ulang form.
    public function store(): void
    {
        $result = $this->service->create($_POST);
```

```
        if ($result['success']) {
            $this->redirect('/mahasiswa', $result['message']);
        }

        // Gagal: tampilkan flash message error + pesan per field, isian
lama tetap terisi
        setFlash('danger', $result['message']);
        $this->tampilForm($result['data'], url('/mahasiswa/store'),
$result['errors'], $_POST);
    }

    public function edit(): void
    {
        try {
            $m = $this->service->find((int) ($_GET['id'] ?? 0));
        } catch (Throwable $e) {
            Logger::error('Gagal memuat data mahasiswa untuk diedit', $e);
            $this->redirect('/mahasiswa', 'Data mahasiswa gagal dimuat.
Silakan coba lagi nanti.', 'danger');
        }

        if ($m === null) {
            $this->redirect('/mahasiswa', 'Data mahasiswa tidak ditemukan.',
'danger');
        }

        $this->tampilForm($m, url('/mahasiswa/update'));
    }

    public function update(): void
    {
        $result = $this->service->update((int) ($_POST['id'] ?? 0), $_POST);

        if ($result['success']) {
            $this->redirect('/mahasiswa', $result['message']);
        }

        // data null = tidak ada form yang bisa ditampilkan (mis. data tidak
ditemukan)
        if ($result['data'] === null) {
            $this->redirect('/mahasiswa', $result['message'], 'danger');
        }

        setFlash('danger', $result['message']);
        $this->tampilForm($result['data'], url('/mahasiswa/update'),
$result['errors'], $_POST);
    }
```

```
    public function delete(): void
    {
        $result = $this->service->delete((int) ($_POST['id'] ?? 0));

        $this->redirect('/mahasiswa', $result['message'], $result['success']
? 'success' : 'danger');
    }

    private function tampilForm(Mahasiswa $m, string $action, array $errors
= [], array $old = []): void
    {
        try {
            $prodiList = $this->service->daftarProdi();
        } catch (Throwable $e) {
            Logger::error('Gagal memuat daftar prodi', $e);
            setFlash('danger', 'Daftar program studi gagal dimuat. Silakan
coba lagi nanti.');
            $prodiList = [];
        }

        $this->view('mahasiswa/form', [
            'm'         => $m,
            'action'    => $action,
            'errors'    => $errors,
            'old'       => $old,
            'prodiList' => $prodiList,
        ]);
    }
}
```

app/Models/BaseModel.php

```
<?php

require_once __DIR__ . '/../Database.php';

// Induk semua class yang butuh akses database (Inheritance).
// Koneksi PDO diambil dari objek Database yang disuntikkan lewat
constructor,
// sehingga class turunan cukup memakai $this->pdo.
abstract class BaseModel
{
    protected PDO $pdo;

    public function __construct(Database $db)
    {
        $this->pdo = $db->getConnection();
    }
}
```

app/Models/Mahasiswa.php

```php
<?php

class Mahasiswa
{
    // Semua atribut private: hanya bisa diakses lewat getter dan setter
    private int $id = 0;
    private string $nim = '';
    private string $nama = '';
    private string $email = '';
    private int $prodiId = 0;
    private int $angkatan;
    private string $prodiNama = '';   // hanya untuk tampilan (hasil JOIN)

    public function __construct()
    {
        $this->angkatan = (int) date('Y');
    }

    // Buat objek dari satu baris data database (data dari database dianggap sudah valid)
    public static function dariBaris(array $row): self
    {
        $m = new self();
        $m->id        = (int) $row['id'];
        $m->nim       = $row['nim'];
        $m->nama      = $row['nama'];
        $m->email     = $row['email'];
        $m->prodiId   = (int) $row['prodi_id'];
        $m->angkatan  = (int) $row['angkatan'];
        $m->prodiNama = $row['prodi_nama'] ?? '';
        return $m;
    }

    // ---------- Getter ----------
    public function getId(): int          { return $this->id; }
    public function getNim(): string      { return $this->nim; }
    public function getNama(): string     { return $this->nama; }
    public function getEmail(): string    { return $this->email; }
    public function getProdiId(): int     { return $this->prodiId; }
    public function getAngkatan(): int    { return $this->angkatan; }
    public function getProdiNama(): string { return $this->prodiNama; }

    // ---------- Setter (dengan validasi sederhana) ----------
    public function setId(int $id): void
    {
        $this->id = $id;
```

```
    }

    public function setNim(string $nim): void
    {
        $nim = trim($nim);

        if ($nim === '') {
            throw new InvalidArgumentException('NIM wajib diisi.');
        }
        if (!preg_match('/^[A-Za-z0-9]+$/', $nim)) {
            throw new InvalidArgumentException('NIM hanya boleh berisi huruf
dan angka.');
        }
        if (strlen($nim) > 20) {
            throw new InvalidArgumentException('NIM maksimal 20 karakter.');
        }

        $this->nim = $nim;
    }

    public function setNama(string $nama): void
    {
        $nama = trim($nama);

        if ($nama === '') {
            throw new InvalidArgumentException('Nama tidak boleh kosong.');
        }
        if (mb_strlen($nama) > 100) {
            throw new InvalidArgumentException('Nama maksimal 100
karakter.');
        }

        $this->nama = $nama;
    }

    public function setEmail(string $email): void
    {
        $email = trim($email);

        if ($email === '') {
            throw new InvalidArgumentException('Email wajib diisi.');
        }
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException('Format email tidak valid.');
        }
        if (mb_strlen($email) > 100) {
            throw new InvalidArgumentException('Email maksimal 100
karakter.');
        }
```

```
        $this->email = $email;
    }

    public function setProdiId(int $prodiId): void
    {
        if ($prodiId <= 0) {
            throw new InvalidArgumentException('Pilih program studi.');
        }

        $this->prodiId = $prodiId;
    }

    public function setAngkatan(int $angkatan): void
    {
        if ($angkatan < 1990 || $angkatan > (int) date('Y') + 1) {
            throw new InvalidArgumentException('Angkatan tidak valid.');
        }

        $this->angkatan = $angkatan;
    }
}
```

app/Repositories/MahasiswaRepository.php

```
<?php

require_once __DIR__ . '/../Models/BaseModel.php';
require_once __DIR__ . '/../Models/Mahasiswa.php';

// Repository Pattern: seluruh query SQL mahasiswa ada di class ini.
// Mewarisi BaseModel, jadi $this->pdo sudah tersedia dari class induk
// (Database disuntikkan lewat constructor BaseModel).
class MahasiswaRepository extends BaseModel
{
    // Semua mahasiswa + nama prodi (JOIN). Jika $keyword diisi, cari nama
atau NIM dengan LIKE
    public function all(string $keyword = ''): array
    {
        $sql = "SELECT m.*, p.nama AS prodi_nama
                FROM mahasiswa m
                JOIN prodi p ON m.prodi_id = p.id";
        $params = [];

        if ($keyword !== '') {
            $sql .= " WHERE m.nama LIKE :nama OR m.nim LIKE :nim";
            $params['nama'] = '%' . $keyword . '%';
            $params['nim']  = '%' . $keyword . '%';
        }
```

```
        $sql .= " ORDER BY m.nim";

        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);

        $hasil = [];
        foreach ($stmt->fetchAll() as $row) {
            $hasil[] = Mahasiswa::dariBaris($row);
        }
        return $hasil;
    }

    public function find(int $id): ?Mahasiswa
    {
        $stmt = $this->pdo->prepare("SELECT * FROM mahasiswa WHERE id =
:id");
        $stmt->execute(['id' => $id]);
        $row = $stmt->fetch();

        return $row ? Mahasiswa::dariBaris($row) : null;
    }

    public function create(Mahasiswa $m): void
    {
        $stmt = $this->pdo->prepare(
            "INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan)
             VALUES (:nim, :nama, :email, :prodi_id, :angkatan)"
        );
        $stmt->execute([
            'nim'      => $m->getNim(),
            'nama'     => $m->getNama(),
            'email'    => $m->getEmail(),
            'prodi_id' => $m->getProdiId(),
            'angkatan' => $m->getAngkatan(),
        ]);
    }

    public function update(Mahasiswa $m): void
    {
        $stmt = $this->pdo->prepare(
            "UPDATE mahasiswa
             SET nim = :nim, nama = :nama, email = :email, prodi_id =
:prodi_id, angkatan = :angkatan
             WHERE id = :id"
        );
        $stmt->execute([
            'nim'      => $m->getNim(),
            'nama'     => $m->getNama(),
```

```
            'email'    => $m->getEmail(),
            'prodi_id' => $m->getProdiId(),
            'angkatan' => $m->getAngkatan(),
            'id'       => $m->getId(),
        ]);
    }

    public function delete(int $id): void
    {
        $stmt = $this->pdo->prepare("DELETE FROM mahasiswa WHERE id = :id");
        $stmt->execute(['id' => $id]);
    }

    // Cek apakah NIM sudah dipakai mahasiswa lain.
    // $exceptId diisi saat edit, supaya NIM milik data itu sendiri tidak
dianggap duplikat.
    public function existsByNim(string $nim, int $exceptId = 0): bool
    {
        $stmt = $this->pdo->prepare("SELECT 1 FROM mahasiswa WHERE nim =
:nim AND id <> :id");
        $stmt->execute(['nim' => $nim, 'id' => $exceptId]);

        return (bool) $stmt->fetchColumn();
    }
}
```

app/Repositories/ProdiRepository.php

```
<?php

require_once __DIR__ . '/../Models/BaseModel.php';

// Repository khusus tabel prodi.
// Sebelumnya daftarProdi() ada di MahasiswaRepository; sekarang dipisah
// supaya setiap Repository hanya mengurus satu tabel.
class ProdiRepository extends BaseModel
{
    // Daftar prodi untuk pilihan di form
    public function all(): array
    {
        return $this->pdo->query("SELECT id, kode, nama FROM prodi ORDER BY
kode")->fetchAll();
    }

    // Dipakai Service untuk memastikan prodi yang dipilih benar-benar ada
    public function exists(int $id): bool
    {
        $stmt = $this->pdo->prepare("SELECT 1 FROM prodi WHERE id = :id");
        $stmt->execute(['id' => $id]);
```

```
        return (bool) $stmt->fetchColumn();
    }
}
```

app/Services/MahasiswaService.php

```php
<?php

require_once __DIR__ . '/../Logger.php';
require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';
require_once __DIR__ . '/../Repositories/ProdiRepository.php';

// Service Layer: tempat logika bisnis mahasiswa.
//   Controller  -> mengatur alur (terima request, tentukan response)
//   Service     -> validasi + aturan bisnis + exception handling + logging
(class ini)
//   Repository  -> menjalankan query ke database (prepared statement)
class MahasiswaService
{
    // Kode error MySQL untuk "Duplicate entry"
    private const MYSQL_DUPLICATE = 1062;

    // Constructor injection: kedua Repository diberikan dari luar (lihat
public/index.php)
    public function __construct(
        private MahasiswaRepository $repo,
        private ProdiRepository $prodiRepo
    ) {}

    // ---------- Proses baca data (diteruskan ke Repository) ----------
    // Exception database tidak ditangkap di sini; Controller yang
menangkapnya
    // supaya bisa menentukan tampilan yang aman bagi pengguna.

    public function all(string $keyword = ''): array
    {
        return $this->repo->all($keyword);
    }

    public function find(int $id): ?Mahasiswa
    {
        return $this->repo->find($id);
    }

    public function daftarProdi(): array
    {
        return $this->prodiRepo->all();
    }
```

```php
    // ---------- Proses tulis data ----------
    // Semua method di bawah mengembalikan array:
    //   success (bool), message (string, untuk flash message),
    //   errors (array per field), data (objek Mahasiswa untuk mengisi ulang
form;
    //   null jika tidak ada form yang perlu ditampilkan, mis. data tidak
ditemukan)

    public function create(array $input): array
    {
        $m = null;

        try {
            [$m, $errors] = $this->validate($input);

            if (!empty($errors)) {
                return $this->gagal($m, $errors);
            }

            $this->repo->create($m);
        } catch (PDOException $e) {
            return $this->gagalDatabase($e, $m ?? new Mahasiswa(),
'menambah');
        }

        return $this->sukses($m, 'Data mahasiswa berhasil ditambahkan.');
    }

    public function update(int $id, array $input): array
    {
        $m = null;

        try {
            if ($this->repo->find($id) === null) {
                return $this->tidakDitemukan();
            }

            [$m, $errors] = $this->validate($input, $id);

            if (!empty($errors)) {
                return $this->gagal($m, $errors);
            }

            $this->repo->update($m);
        } catch (PDOException $e) {
            $kosong = new Mahasiswa();
            $kosong->setId($id);
```

```
            return $this->gagalDatabase($e, $m ?? $kosong, 'mengubah');
        }

        return $this->sukses($m, 'Data mahasiswa berhasil diubah.');
    }

    public function delete(int $id): array
    {
        try {
            if ($this->repo->find($id) === null) {
                return $this->tidakDitemukan();
            }

            $this->repo->delete($id);
        } catch (PDOException $e) {
            Logger::error("Gagal menghapus mahasiswa id=$id", $e);

            return ['success' => false, 'message' => 'Data gagal dihapus.',
'errors' => [], 'data' => null];
        }

        return ['success' => true, 'message' => 'Data mahasiswa berhasil
dihapus.', 'errors' => [], 'data' => null];
    }

    // ---------- Validasi ----------

    // Mengembalikan [objek Mahasiswa, array error].
    // $id = 0 untuk data baru, > 0 untuk data yang sedang diubah.
    private function validate(array $input, int $id = 0): array
    {
        $m = new Mahasiswa();
        $m->setId($id);
        $errors = [];

        // 1. Aturan format: dicek lewat setter pada class Mahasiswa
        $fields = [
            'nim'      => ['setNim',      (string) ($input['nim'] ?? '')],
            'nama'     => ['setNama',     (string) ($input['nama'] ?? '')],
            'email'    => ['setEmail',    (string) ($input['email'] ?? '')],
            'prodi_id' => ['setProdiId',  (int) ($input['prodi_id'] ?? 0)],
            'angkatan' => ['setAngkatan', (int) ($input['angkatan'] ?? 0)],
        ];

        foreach ($fields as $field => [$setter, $nilai]) {
            try {
                $m->$setter($nilai);
            } catch (InvalidArgumentException $e) {
                $errors[$field] = $e->getMessage();
```

```
            }
        }

        // 2. Aturan bisnis: butuh data dari database, jadi dicek di Service
        if (!isset($errors['nim']) && $this->repo->existsByNim($m->getNim(),
$id)) {
            $errors['nim'] = 'NIM sudah terdaftar.';
        }

        if (!isset($errors['prodi_id']) && !$this->prodiRepo->exists($m-
>getProdiId())) {
            $errors['prodi_id'] = 'Program studi tidak tersedia.';
        }

        return [$m, $errors];
    }

    // ---------- Penyusun hasil ----------

    private function sukses(Mahasiswa $m, string $message): array
    {
        return ['success' => true, 'message' => $message, 'errors' => [],
'data' => $m];
    }

    private function tidakDitemukan(): array
    {
        return ['success' => false, 'message' => 'Data mahasiswa tidak
ditemukan.', 'errors' => [], 'data' => null];
    }

    // Hasil gagal. Pesan flash dibedakan agar pengguna tahu penyebabnya.
    private function gagal(Mahasiswa $m, array $errors): array
    {
        $message = 'Data gagal disimpan. Periksa kembali isian form.';

        if (($errors['nim'] ?? '') === 'NIM sudah terdaftar.') {
            $message = 'NIM sudah terdaftar.';
        }

        return ['success' => false, 'message' => $message, 'errors' =>
$errors, 'data' => $m];
    }

    // Exception database: detail teknis HANYA ke app.log, pengguna mendapat
pesan aman.
    private function gagalDatabase(PDOException $e, Mahasiswa $m, string
$aksi): array
    {
```

```
        Logger::error("Gagal $aksi data mahasiswa", $e);

        // NIM duplikat yang lolos pengecekan existsByNim (mis. dua request
bersamaan)
        if (($e->errorInfo[1] ?? 0) === self::MYSQL_DUPLICATE) {
            return $this->gagal($m, ['nim' => 'NIM sudah terdaftar.']);
        }

        return ['success' => false, 'message' => 'Data gagal disimpan.',
'errors' => [], 'data' => $m];
    }
}
```

app/Views/header.php

```
<?php
$flash = getFlash();   // flash message: tampil sekali lalu hilang
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>SI Akademik</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body class="bg-light">

<nav class="navbar navbar-expand navbar-dark bg-dark mb-4">
    <div class="container">
        <span class="navbar-brand">SI Akademik</span>
        <div class="navbar-nav">
            <a class="nav-link" href="<?= url('/mahasiswa')
?>">Mahasiswa</a>
        </div>
        <div class="ms-auto d-flex align-items-center">
            <span class="navbar-text me-3"><?= e($_SESSION['user_name'] ??
'') ?></span>
            <a class="btn btn-outline-light btn-sm" href="<?= url('/logout')
?>">Logout</a>
        </div>
    </div>
</nav>

<main class="container pb-5">
    <?php if ($flash): ?>
```

```
        <?php $ikon = $flash['type'] === 'success' ? '✓ ' : ($flash['type']
=== 'danger' ? '✕ ' : ''); ?>
        <div class="alert alert-<?= e($flash['type']) ?>"><?= $ikon .
e($flash['message']) ?></div>
    <?php endif; ?>
```

app/Views/auth/login.php

```
<?php
$flash = getFlash();   // flash message: tampil sekali lalu hilang
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Login - SI Akademik</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body class="bg-light">

<div class="container" style="max-width: 420px; margin-top: 10vh;">
    <div class="card shadow-sm">
        <div class="card-body p-4">
            <h1 class="h4 mb-3 text-center">SI Akademik</h1>

            <?php if ($flash): ?>
                <div class="alert alert-<?= e($flash['type']) ?>"><?=
e($flash['message']) ?></div>
            <?php endif; ?>

            <form method="POST" action="<?= url('/login') ?>">
                <div class="mb-3">
                    <label class="form-label">Username</label>
                    <input type="text" name="username" class="form-control"
required autofocus>
                </div>
                <div class="mb-3">
                    <label class="form-label">Password</label>
                    <input type="password" name="password" class="form-
control" required>
                </div>
                <button type="submit" class="btn btn-primary w-
100">Login</button>
            </form>
        </div>
    </div>
```

```
</div>

</body>
</html>
```

app/Views/errors/500.php

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Terjadi Kesalahan - SI Akademik</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body class="bg-light">
<div class="container text-center" style="max-width: 520px; margin-top:
15vh;">
    <h1 class="h3">Terjadi kesalahan pada sistem</h1>
    <p class="text-muted">Permintaan Anda belum dapat diproses. Silakan coba
lagi beberapa saat lagi.</p>
    <a href="<?= url('/mahasiswa') ?>" class="btn btn-primary">Kembali</a>
</div>
</body>
</html>
```

app/Views/mahasiswa/index.php

```
<?php require __DIR__ . '/../header.php'; ?>

<div class="d-flex justify-content-between mb-3">
    <h1 class="h3">Data Mahasiswa</h1>
    <a href="<?= url('/mahasiswa/create') ?>" class="btn btn-primary">+
Tambah Mahasiswa</a>
</div>

<form method="GET" action="<?= url('/mahasiswa') ?>" class="row g-2 mb-3">
    <div class="col-md-6">
        <input type="text" name="q" value="<?= e($keyword) ?>" class="form-
control"
               placeholder="Cari berdasarkan nama atau NIM...">
    </div>
    <div class="col-auto">
        <button type="submit" class="btn btn-outline-primary">Cari</button>
        <?php if ($keyword !== ''): ?>
            <a href="<?= url('/mahasiswa') ?>" class="btn btn-outline-
secondary">Reset</a>
```

```
        <?php endif; ?>
    </div>
</form>

<table class="table table-striped table-bordered bg-white">
    <thead class="table-dark">
        <tr><th>No</th><th>NIM</th><th>Nama</th><th>Email</th><th>Prodi</th>
<th>Angkatan</th><th>Aksi</th></tr>
    </thead>
    <tbody>
        <?php foreach ($daftar as $i => $m): ?>
            <tr>
                <td><?= $i + 1 ?></td>
                <td><?= e($m->getNim()) ?></td>
                <td><?= e($m->getNama()) ?></td>
                <td><?= e($m->getEmail()) ?></td>
                <td><?= e($m->getProdiNama()) ?></td>
                <td><?= e($m->getAngkatan()) ?></td>
                <td>
                    <a href="<?= url('/mahasiswa/edit?id=' . $m->getId())
?>" class="btn btn-sm btn-warning">Edit</a>
                    <form method="POST" action="<?= url('/mahasiswa/delete')
?>" class="d-inline"
                          onsubmit="return confirm('Yakin ingin menghapus
data ini?');">
                        <input type="hidden" name="id" value="<?= $m-
>getId() ?>">
                        <button type="submit" class="btn btn-sm btn-
danger">Hapus</button>
                    </form>
                </td>
            </tr>
        <?php endforeach; ?>
        <?php if (!$daftar): ?>
            <tr><td colspan="7" class="text-center">
                <?= $keyword !== '' ? 'Tidak ada data yang cocok.' : 'Belum
ada data.' ?>
            </td></tr>
        <?php endif; ?>
    </tbody>
</table>

<?php require __DIR__ . '/../footer.php'; ?>
```

app/Views/mahasiswa/form.php

```
<?php
// Variabel: $m (objek Mahasiswa), $action, $prodiList, $errors, $old (input
sebelumnya jika ada error)
```

```php
$invalid = fn(string $f) => isset($errors[$f]) ? 'is-invalid' : '';
require __DIR__ . '/../header.php';
?>

<h1 class="h3 mb-3"><?= $m->getId() ? 'Edit' : 'Tambah' ?> Mahasiswa</h1>

<form method="POST" action="<?= e($action) ?>" class="card card-body"
novalidate>
    <input type="hidden" name="id" value="<?= $m->getId() ?>">

    <div class="mb-3">
        <label class="form-label">NIM</label>
        <input type="text" name="nim" class="form-control <?=
$invalid('nim') ?>"
               value="<?= e($old['nim'] ?? $m->getNim()) ?>">
        <div class="invalid-feedback"><?= e($errors['nim'] ?? '') ?></div>
    </div>

    <div class="mb-3">
        <label class="form-label">Nama</label>
        <input type="text" name="nama" class="form-control <?=
$invalid('nama') ?>"
               value="<?= e($old['nama'] ?? $m->getNama()) ?>">
        <div class="invalid-feedback"><?= e($errors['nama'] ?? '') ?></div>
    </div>

    <div class="mb-3">
        <label class="form-label">Email</label>
        <input type="text" name="email" class="form-control <?=
$invalid('email') ?>"
               value="<?= e($old['email'] ?? $m->getEmail()) ?>">
        <div class="invalid-feedback"><?= e($errors['email'] ?? '') ?></div>
    </div>

    <div class="mb-3">
        <label class="form-label">Program Studi</label>
        <?php $prodiTerpilih = (int) ($old['prodi_id'] ?? $m->getProdiId());
?>
        <select name="prodi_id" class="form-select <?= $invalid('prodi_id')
?>">
            <option value="">-- Pilih Prodi --</option>
            <?php foreach ($prodiList as $p): ?>
                <option value="<?= (int) $p['id'] ?>" <?= $prodiTerpilih ===
(int) $p['id'] ? 'selected' : '' ?>>
                    <?= e($p['kode'] . ' - ' . $p['nama']) ?>
                </option>
            <?php endforeach; ?>
        </select>
```

```
        <div class="invalid-feedback"><?= e($errors['prodi_id'] ?? '')
?></div>
    </div>

    <div class="mb-3">
        <label class="form-label">Angkatan</label>
        <input type="number" name="angkatan" class="form-control <?=
$invalid('angkatan') ?>"
               value="<?= e($old['angkatan'] ?? $m->getAngkatan()) ?>">
        <div class="invalid-feedback"><?= e($errors['angkatan'] ?? '')
?></div>
    </div>

    <div>
        <button type="submit" class="btn btn-primary">Simpan</button>
        <a href="<?= url('/mahasiswa') ?>" class="btn btn-
secondary">Batal</a>
    </div>
</form>

<?php require __DIR__ . '/../footer.php'; ?>
```

config/database.php

```
<?php

return [
    'host'     => 'localhost',
    'dbname'   => 'bkpm_acara_7',
    'username' => 'root',
    'password' => '',
    'charset'  => 'utf8mb4',
];
```

public/.htaccess

```
RewriteEngine On

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [L]
```

public/index.php

```
<?php

// Jangan tampilkan error teknis ke pengguna; error dicatat ke
storage/logs/app.log
ini_set('display_errors', '0');
error_reporting(E_ALL);
```

```php
// Cookie session lebih aman: tidak bisa dibaca JavaScript dan tidak ikut
request lintas situs
session_set_cookie_params(['httponly' => true, 'samesite' => 'Lax']);
session_start();

// BASE_PATH otomatis (contoh: /Bkpm_Acara_13/public), jadi tidak perlu
diubah manual
define('BASE_PATH', rtrim(str_replace('\\', '/',
dirname($_SERVER['SCRIPT_NAME'])), '/'));

require_once __DIR__ . '/../app/helpers.php';
require_once __DIR__ . '/../app/Logger.php';
require_once __DIR__ . '/../app/Controllers/AuthController.php';
require_once __DIR__ . '/../app/Controllers/MahasiswaController.php';

// Penangkap terakhir: exception yang lolos (mis. koneksi database gagal)
// dicatat ke app.log, pengguna hanya melihat halaman error yang aman.
set_exception_handler(function (Throwable $e): void {
    Logger::error('Exception tidak tertangani', $e);
    http_response_code(500);
    require __DIR__ . '/../app/Views/errors/500.php';
});

// Ambil path tanpa BASE_PATH, contoh: /mahasiswa/edit
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$uri = substr($uri, strlen(BASE_PATH));
$uri = '/' . trim($uri, '/');

// Login & logout (boleh diakses tanpa login)
if ($uri === '/login') {
    $auth = new AuthController();
    $_SERVER['REQUEST_METHOD'] === 'POST' ? $auth->login() : $auth-
>loginForm();
    exit;
}

if ($uri === '/logout') {
    (new AuthController())->logout();
}

// Semua halaman di bawah ini wajib login
cekLogin();

if ($uri === '/') {
    redirect('/mahasiswa');
}

// Merakit objek (Dependency Injection):
// Database -> Repository -> Service -> Controller
```

```php
$database    = new Database();
$mhsRepo     = new MahasiswaRepository($database);
$prodiRepo   = new ProdiRepository($database);
$service     = new MahasiswaService($mhsRepo, $prodiRepo);
$controller  = new MahasiswaController($service);

// GET untuk menampilkan, POST untuk mengubah data
$parts  = explode('/', trim($uri, '/'));
$action = $parts[1] ?? 'index';
$allowed = ($_SERVER['REQUEST_METHOD'] === 'POST')
    ? ['store', 'update', 'delete']
    : ['index', 'create', 'edit'];

if ($parts[0] === 'mahasiswa' && count($parts) <= 2 && in_array($action,
$allowed, true)) {
    $controller->$action();
} else {
    http_response_code(404);
    echo '<h1>404</h1><p>Halaman tidak ditemukan.</p>';
}
```

}

storage/logs/app.log

```
2026-10-07 14:02:50 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
2026-10-07 14:02:52 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
2026-10-07 14:02:54 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
2026-10-07 14:02:54 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
2026-10-07 14:02:59 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
2026-10-07 14:03:34 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
2026-10-07 14:03:35 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
2026-10-07 14:03:35 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
```

```
2026-10-07 14:03:36 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
2026-10-07 14:03:36 - [ERROR] Exception tidak tertangani | ParseError:
syntax error, unexpected single-quoted string "username", expecting "]"
(database.php:6)
```

BAB V HASIL DAN PEMBAHASAN

Setelah implementasi Acara 14, aplikasi memiliki pemisahan tanggung jawab yang lebih jelas. Controller mengatur alur request dan response, Service mengatur validasi dan business logic, Repository menangani query database, Logger mencatat kejadian, dan helper/session menangani Flash Message.

5.1 Hasil Pengujian

Fitur	Skenario Pengujian	Hasil yang Diharapkan
Login	Masukkan username dan password	Berhasil masuk atau muncul pesan kesalahan
Tampil data	Membuka halaman mahasiswa	Daftar mahasiswa tampil
Pencarian	Memasukkan NIM/nama	Data sesuai keyword tampil
Tambah data	Mengisi data valid	Data tersimpan dan redirect ke daftar
Validasi	Mengisi data tidak valid	Pesan error ditampilkan pada form
NIM duplikat	Memasukkan NIM yang sudah ada	Data ditolak dan pesan ditampilkan
Edit data	Mengubah data mahasiswa	Data berubah dan redirect ke daftar
Hapus data	Menekan tombol hapus	Data terhapus dan redirect ke daftar
Flash Message	Berhasil/gagal CRUD	Pesan hanya tampil satu kali
Exception	Memicu kegagalan operasi database	Kesalahan ditangani dan tidak merusak alur aplikasi
Logging	Terjadi kejadian/error	Informasi dicatat pada storage/logs/app.log

5.2 Pembahasan

Penerapan Security dilakukan melalui validasi dan prepared statement pada Repository. Exception Handling digunakan agar kesalahan database dapat ditangani secara terkontrol.

Logging membantu proses debugging dengan menyimpan informasi kejadian pada file log. Pola PRG membuat proses POST diikuti redirect sehingga refresh halaman tidak mengirim ulang form. Keseluruhan CRUD tetap mengikuti pembagian Controller-Service-Repository sehingga kode lebih mudah dipelihara.

BAB VI KESIMPULAN

Acara 14 berhasil mengintegrasikan Security, Exception, Logging, PRG, dan CRUD lengkap ke dalam aplikasi Sistem Informasi Akademik. Proses CRUD dipisahkan melalui Controller, Service, dan Repository. Validasi dan penanganan exception meningkatkan keamanan dan kestabilan aplikasi, Logger membantu pencatatan error, sedangkan PRG dan Flash Message membuat alur setelah proses POST lebih baik.